BobbyCoin: A Minimal Viable Distributed Blockchain Cryptocurrency in Go

Ryan Ficklin, Madeline Park
Department of Computer Science and Software Engineering, California Polytechnic State University
San Luis Obispo, CA 93407 USA
(rficklin, mpark41)@calpoly.edu

***Abstract*—This final project examined distributed Peer-to-Peer (P2P) cryptocurrency ("BobbyCoin") through a blockchain transaction ledger. Implemented in Go, communication was handled through RPC for the sake of simulation, although the program is extendable to different communications schemes as well. Membership was handled through a Gossip/Heartbeat protocol. Consensus is determined with a blockchain proof-of-work scheme. Fault tolerance is achieved with an easily checkable hashed blockchain ledger of transactions which can be shared, updated, lost, and modified so long as the majority ($> 50\%$) of computing nodes can communicate with one another and identify the current longest blockchain (which represents the largest proof of work contribution).**

***Index Terms*—distributed systems, blockchain, cryptography**

I. Introduction

In this final project we implemented the necessary cryptographic infrastructure to support a distributed blockchain ledger for cryptocurrency exchange among computing nodes. This infrastructure was built on top of the distributed system designed over the course of the lab projects during CSC 569-01-2264. We based our implementation on the specifications detailed in Nakamoto's original white paper [1], with our specific design choices detailed in Section II. There have been multiple simplifying assumptions made during implementation for the sake of time, all of which are detailed in Section III. We claim that our distributed system design can be scaled for peer-to-peer use despite these simplifying assumptions.

A. Division of Labor

All new code implemented on top of the existing lab project code base was programmed by hand in a pair-programming style between the authors. Contributions are considered to be an equal effort from both parties, with two pairs of eyes observing the code written for nearly the entire course of the final project.

II. Design Description

A. Peer to Peer Communication

Computing nodes are initialized with a `Node` struct containing:

- ID : `<integer>`
- Public Key: `<[]byte>`
- Heartbeat Counter: `<integer>`
- Timestamp: `<time.Time>`

Problem	Technique	Advantage
Membership Discovery and P2P Communication	Gossip/Heartbeat Protocol	Nodes can share their blockchain ledger with strong probability that the entire network will be able to witness their proof of work, despite not having direct communication with all available computing nodes
Fault Tolerance	Hashed Blockchain Ledger	Any node dying in the network and rejoining must only recompute the validity longest blockchain that is communicated to them in order to have sufficient change to begin mining and transacting as normal
Consensus	Proof of Work	25 leading 0 bits in a SHA256 hash represents an exponentially difficult problem that guarantees that any node which offers a new block to be minted (added to the longest blockchain) can be easily verified and accepted by all other computing nodes. No distributed consensus algorithm like RAFT or Paxos is required, as proof of work favors computational resources instead of other distributed-system-theoretic schemes

Fig. 1. A brief outline of the distributed systems problems and solutions demonstrated by this project.

- Alive Flag: `<boolean>`
- Blockchain: `<[]Block>`
- Unverified Transaction Pool: `<[]Transaction>`

Definitions for `Block` and `Transaction` will appear momentarily. This is all of the outward-facing information a computing node has in our P2P network. Additionally, nodes internally contain:

- Private Key : `<ecdsa.PrivateKey>`
- Spent Transaction Pool: `<[]Transaction>`

The private key corresponds with the public key present in the `Node` struct, following Go's `crypto/ecdsa` package for Elliptic Curve Digital Signature Algorithms, with the NIST P-256 Elliptic Curve Algorithm for Go's `crypto/elliptic` package. Other public/private key schemes and algorithms are not supported on our blockchain protocol.

Our communication between computing nodes is performed by an RPC server with a send/receive mailbox messaging system. Nodes can share their current state with 3 neighbors (determined randomly at client startup), who can in turn share with their own 3 neighbors, fanning out until our network of up to 8 nodes can probabilistically communicate All-to-All. This protocol is our Gossip/Heartbeat protocol, wherein Nodes have a membership table which they exchange with neighbors, constantly sending heartbeats and updating membership status. This gossiping also serves as the method through which to share new transactions and minted blocks on the blockchain, discussed later. We decided this for our network topology for two primary reasons. The first is that it was the topology used during the labs, and was thus convenient as it was already implemented. However, we believe that the choice represents a strong simulation of real P2P networks, wherein not all computing nodes will be part of the same network, and thus must communicate to neighbors who have neighbors (and so on) until a network edge device can bridge the gap to a new neighborhood.

We are aware that this network topology, as well as true P2P schemes, would not require an ID number for a node. Public Keys are unique and would function adequately as an ID without need for any ID distribution scheme. However, the ID scheme in previous labs was fairly load-bearing in the Gossip/Heartbeat protocol, and thus would be an unnecessary engineering effort to tear out and replace with public keys instead. Hence, a necessary modification for transitioning this blockchain system from RPC to full P2P communication would require both a different networking protocol, as well as a transition away from ID into full Public Key identification.

B. Blockchain Protocol

Acknowledgment: This blockchain protocol is based off of specifications from Professor Zachary Peterson's CSC 323 Lab 4: Minimum Viable Blockchain. Both authors have taken this course as an exploration into cryptography engineering, and desired to tackle the system from the perspective of P2P distributed systems. Additionally, all CSC 323 Lab code was written in Python, and our implementation is written entirely in Go. So, although design decisions were similar, engineering effort and system implementation was a unique and dedicated experience tailored toward CSC 569 course learning objectives. Also, CSC 323 did not involve coordinating or simulating P2P network communication between computing nodes, which, as described in the previous subsection, was an additional and sophisticated consideration in system design.

A `Block` is specified as follows:

- Block ID: `<[]byte>`
- Nonce: `<[]byte>`
- Proof of Work: `<[]byte>`
- Previous Block Id: `<[]byte>`
- Transaction: `<Transaction>`

A `Transaction` is:

- Signature: `<[]byte>`
- Input: `<TX_Input>`
- Output: `<[]TX_Output>`

`TX_Inputs` contain:

- Block ID: `<[]byte`
- Output Number: `<integer>`

`TX_Outputs` contain:

- Coin Value: `<integer>`
- Public Key : `<[]byte>`

Note: Most structs contain byte arrays as the majority of their types. Our structures were handled with Go's structs rather than an serialized format such as JSON. However, RPC communication cannot handle complex Go values, such as `ecdsa.PublicKey`, so we ended up serializing the majority of our values anyways (oftentimes by using built-in functions for marshaling into JSON).

The ID of block is the SHA256 hash of the serialized transaction. Each block has a nonce value, which, when serialized and combined with the transaction and previous block ID, is hashed once again using SHA256 in order to calculate a proof of work. The proof of work must have a set number of leading 0s in its bit representation, 23 zeros in our case, in order to be valid and added to the block chain.

Nodes will trust any blockchain longer than their current once so long as the hard-coded genesis block (first block in the blockchain) is correct, all proof of works can correctly be verified to pass the difficulty, all transactions are valid on each block are valid, and each block correctly points to the block id of the previous block. This works as a consensus algorithm without a scheme like RAFT or Paxos because Proof of Work favors computational effort (so long as it is performed correctly) over any other metric. Nodes, so long as they believe that the majority of the computational power is held by honest nodes with which they are communicating, can both trust and verify that consensus has been reached on the current longest blockchain containing the proof of the most work performed.

"BobbyCoins", as they are called in our cryptocurrency system, are measured by the values from the set of outputs of each transaction. If a computing node has a transaction on the block chain in which their public key appears with an associated coin value in the set of that transaction's outputs, they may send up to that number of coins to another computing nodes public key, with the remaining coins sent back to themselves. The only caveat is that the output index of that transaction from that block can only be used once as an input to a new transaction. This is what prevents double spending of coins. Once again, proof of work contains the inputs and outputs of each block's transaction, so no node may forge a double spend for themselves without having overwhelming computational power across the network to stay ahead of all the other coin miners.

Transactions have a digital signature registered with them, signed by the private key of node that is used as the input of that transaction. No node knows the private key of another, thus they cannot realistically create a transaction that can be verified with a public key other than their own. In our actual

system, for the purposes of consistent simulation, private keys are stored in plaintext in a file readable by any node. However, there is no reason that our scheme would not work generating public/private key pairs on their own, secret from all other computing nodes.

The final output on each transaction for each block is the coinbase, an incentive for mining and minting coins on our blockchain. The miner performing proof of work may add a single additional output transaction to giving 50 BobbyCoins to their own public key. This is the only method for generating new coins, but are spendable as in any other transaction. Coinbases of incorrect values will not be minted as they will not be verified and validated by other nodes.

III. Limitations and Assumptions

There are a couple of optimizations in [1] which we forgo in our implementation. We do not have a continuously changing proof of work difficulty that scales with the computational resources available in our entire network. This is acceptable for our purposes because our simulation itself has finite computational resources, and can only support 8 existing nodes.

Similarly, we do not have diminishing coinbase values with the more coins that are minted. Our simulation starts with only a genesis block, and the 23-bit difficulty proof of work is slow enough such that any sufficient demonstration of our system design would not effectively need diminishing returns, once again due to our fixed computational output.

We do not implement or use a Merkle tree for storing the blockchain for more efficient storage and checking of blockchain modification. Our blockchains remain small enough that they can be stored and verified comfortably in the memory of the computing nodes.

The biggest usability gap in our blockchain is the limitation of a single transaction per block and single input per transaction. This does make a complex system far easier to implement. However, it prevents nodes from sending more than 50 BobbyCoins to one another at a time. Any transaction which sends less than that number of coins to a public key can then only be used to input that diminished number of coins for the next transaction. Because the BobbyCoins are stored as integers, there is a cap of 1 BobbyCoin as the lowest unit of tradable currency. Even if a computing node only has valid transactions wherein they receive a single BobbyCoin, they can still perform arbitrary payments. They can do this my creating multiple new transactions that give a single BobbyCoin to their desired recipient, until they have created enough transactions to satisfy the agreed upon amount their recipient wished to receive. This ‘hack’ to allow payments in the worst case (i.e. a node is bottle-necked to only be able to move 1 BobbyCoin at a time), would be made easier if all of these transactions could be minted on a single block. However, our design only accounted for single-block, single-transaction semantics. This would be the first optimization to make were we to have time to expand the usability of our system.

Nodes choose blocks to mine from the transaction pool arbitrarily. We chose a first-transaction-come, first-transaction-served scheme. There are no transaction fees, because transactions by their construction can only have outputs exactly equal to the value of the input, plus the value of the coinbase for the miner (with the desired value going to the recipient, all leftover automatically going straight back to the sender). This functionality is not needed because our block chain already has mining incentives through the coinbase system.

IV. Conclusion

This final project was a wonderful intersection between distributed systems and cybersecurity/cryptography. We enjoyed revisiting and upgrading our former blockchain schemes from courses past and viewing them from a new lens: a distributed systems lens. Additionally, after learning difficulties of implementing RAFT and understanding Paxos for consensus algorithms, the elegance of proof of work (despite its disastrous real world impacts and environmental consequences) was fully motivated as a secure P2P consensus mechanism. Combining our blockchain protocol into a fully functional distributed P2P communication simulation sufficiently elevated our understanding of cryptography and P2P distributed systems.

References

[1] S. Nakamoto, “Bitcoin: A peer-to-peer electronic cash system,” 2008, [Accessed: 06-Jun-2026]. [Online]. Available: https://bitcoin.org/bitcoin.pdf